This is CS50
Week 6

Kelly Ding (she/her)

Preceptor

kelly@cs50.harvard.edu

Part 0

C to Python

C

- **Low**-level programming language
- **Manual** memory management with `malloc`, `free`
- Must declare types **explicitly** (`int, char`)

Python

- **High**-level programming language
- **Automatic** memory management with garbage collection
- **Dynamically** typed, interpreter figures it out

Why might CS50 teach **C** first?

Part 1

Python Syntax

```c
char *name = get_string("Name: ");
```

```python
name = input("Name: ")
```

```c
if (strcmp(name, "Alice") == 0)
{
    printf("Hi, %s!\n", name);
}
```

```python
if name == "Alice":
    print(f"Hi, {name}!")
```

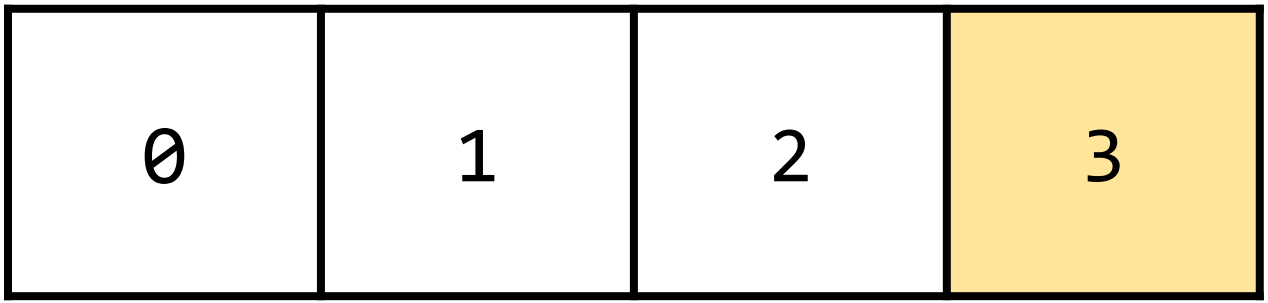
Lists

```
int list[3] = [0, 1, 2];
```

```
list = [0, 1, 2]
```

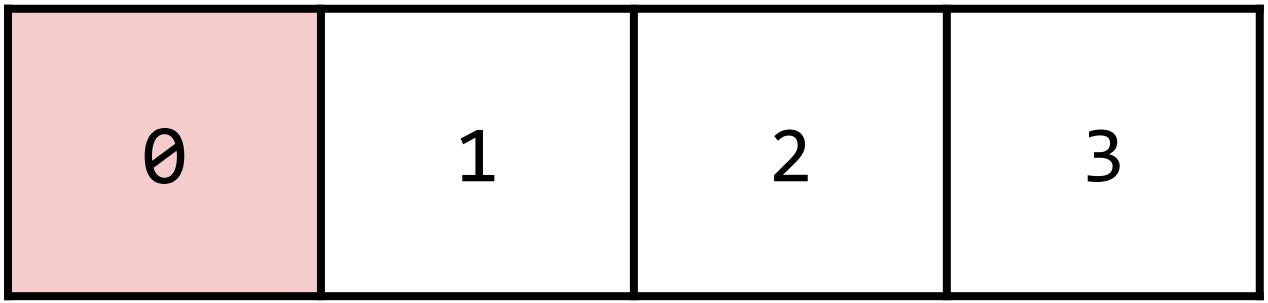
```
list.append(3)
```

```
list
```



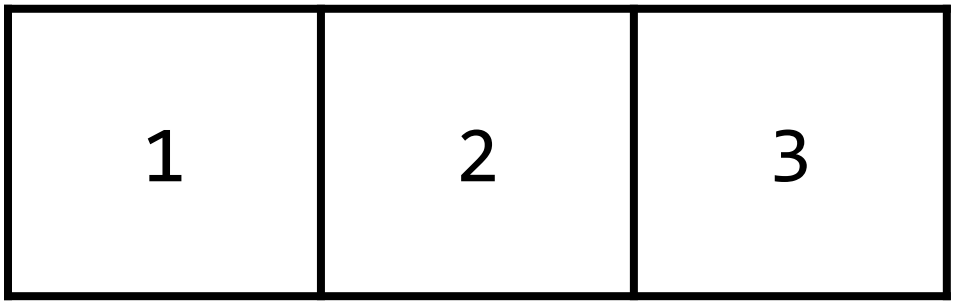
```
list.remove(0)
```

```
list
```



```
list.remove(0)
```

```
list
```



Loops

```
for i in [0, 1, 2]:
    print(i)
```

```
for i in range(3):
    print(i)
```

Default values

```
for i in range(0, 3, 1):
    print(i)
```

Start (inclusive)

```
for i in range(0, 3, 1):
    print(i)
```

End (exclusive)

```
for i in range(0, 3, 1):
    print(i)
```

Step

```
for i in range(0, 3, 1):
    print(i)
```

```
for (int i = 0; i < 3; i++)
{
   print(i)
}
```

```
for i in range(0, 3, 1):
    print(i)
```

```
for i in [0, 1, 2]:
    print(i)

for i in range(0, 3, 1):
    print(i)

for i in range(3):
    print(i)
```

Practice

What do each of the following look like as lists?

For example, `range(3) = [0, 1, 2]`

```
a = range(5)
b = range(1, 3, 2)
c = range(5, 0, -1)
d = range(1, 6, 2)
```

```
int n = 0
while (n < 3)
{
   print(i);
   n++;
}
```

```
n = 0
while (n < 3):
   print(i)
   n += 1
```

String Splicing

```
phrase = "wahoo"
phrase[0] = "w"
phrase[1:3:1] = "ah"
```

```
phrase = "wahoo"
phrase[0] = "w"
phrase[1:3:1] = "ah"
```

Start (inclusive)

`0` by default

```
phrase = "wahoo"
phrase[0] = "w"
phrase[1:3:1] = "ah"
```

End (exclusive)

`len()` by default

```
phrase = "wahoo"
phrase[0] = "w"
phrase[1:3:1] = "ah"
```

Step
`1` by default

Including Libraries

```
#include <cs50.h>
```

```
import cs50
```

```
from cs50 import get_int
```

Declaring Functions

```
int f(int x)
{
    return x;
}
```

```
def f(x):
    return x
```

Command-line Arguments

```
int main(int argc, char *argv[])
{
    if (argc > 1)
    {
        print(argv[1]);
    }
}
```

```
import sys

def main():
    if len(sys.argv) > 1:
        print(sys.argv[1])
```

Note: Python does not have a direct equivalent to **`argc`**

String Predictions

Download and open str_prediction.py in cs50.dev.

On your own, predict the outcomes for each "Round" of string manipulation in Python. Write your predictions **without** running **str_prediction.py**. Then, run **python str_prediction.py** to see what's actually happening!

Afterwards, try modifying the range function in Round 6 to print a string backwards.

Part 2

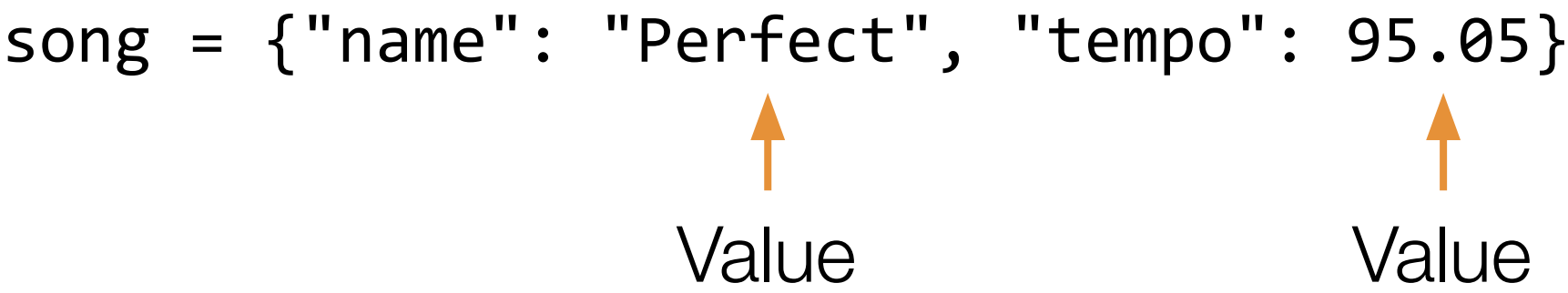
Dictionaries

```
song = {"name": "Perfect", "tempo": 95.05}
```

```
song = {"name": "Perfect", "tempo": 95.05}
```

Key

Key


song = {"name": "Perfect", "tempo": 95.05}
Value
Value

```
song = {"name": "Perfect", "tempo": 95.05}
```

song	
"name"	"Perfect"
"tempo"	95.05

```
song["name"]
```

song	
"name"	"Perfect"
"tempo"	95.05

```
song["tempo"] = "100"
```

song	
"name"	"Perfect"
"tempo"	100

```
song["album"] = "Divide"
```

song	
"name"	"Perfect"
"tempo"	100
"album"	"Divide"

```
song.keys() → ["name", "tempo", "album"]
song.values() → ["Perfect", 0, "Divide"]
```

song	
"name"	"Perfect"
"tempo"	0
"album"	"Divide"

songs

```
[{name: "Perfect", tempo: 95.05},
 {name: "Eastside", tempo: 89.391},
 {name: "Wolves", tempo: 124.946},
 {name: "Him & I", tempo: 87.908}]
```

songs

```
[{name: "Perfect", tempo: 95.05},
 {name: "Eastside", tempo: 89.391},
 {name: "Wolves", tempo: 124.946},
 {name: "Him & I", tempo: 87.908}]
```

songs

```
[{name: "Perfect", tempo: 95.05},
 {name: "Eastside", tempo: 89.391},
 {name: "Wolves", tempo: 124.946},
 {name: "Him & I", tempo: 87.908}]
```

songs

```
[{name: "Perfect", tempo: 95.05},
 {name: "Eastside", tempo: 89.391},
 {name: "Wolves", tempo: 124.946},
 {name: "Him & I", tempo: 87.908}]
```

How do you reference the tempo of "Wolves"?

2018_top100.csv

```
name,tempo
God's Plan,77.169
SAD!,75.023
rockstar (feat. 21 Savage),159.847
Psycho (feat. Ty Dolla $ign),140.124
In My Feelings,91.03
Better Now,145.028
...
```

```
with open(2018_top100.csv) as file:
```

```
with open(2018_top100.csv) as file:
    file_reader = csv.DictReader(file)
```

`file_reader` **returns**

```
[
    {"name": "God's Plan", "tempo": "77.169"},
    {"name": "SAD!, "tempo": "75.023"},
    {"name": "rockstar (feat. 21 Savage)", "tempo": "159.847"},
    {"name": "Psycho (feat. Ty Dolla $ign)", "tempo": "140.124"},
    …
]
```

`file_reader.fieldnames` **returns** `["name", "tempo"]`

```
with open(2018_top100.csv) as file:
    file_reader = csv.DictReader(file)
    for row in file_reader:
        ...
```

```
[
    {"name": "God's Plan", "tempo": "77.169"},
    {"name": "SAD!, "tempo": "75.023"},
    {"name": "rockstar (feat. 21 Savage)", "tempo": "159.847"},
    {"name": "Psycho (feat. Ty Dolla $ign)", "tempo": "140.124"},
    …
]
```

```
[
    {"name": "God's Plan", "tempo": "77.169"},
    {"name": "SAD!, "tempo": "75.023"},
    {"name": "rockstar (feat. 21 Savage)", "tempo":
    "159.847"},
    {"name": "Psycho (feat. Ty Dolla $ign)", "tempo":
    "140.124"},
    …
]
```

Crafting Playlists

Download playlist.py and 2018_top100.csv.

Complete the TODOs in **playlist.py** so that a user can build a playlist of popular songs within a tempo range.

If feeling more comfortable, try allowing the user to eliminate songs with certain words or phrases in the title (e.g., "love").

This is CS50
Week 6